Multi-Threading in Java

何明昕 He Mingxin, Max

Operating Systems

2026 Spring

Definitions of Threads and Processes

- Threads in multithreaded processors may or may not be same as software threads

- Process:
  - An instance of program running on computer

- Thread: dispatchable unit of work within process
  - Includes processor context (which includes the program counter and stack pointer) and data area for stack
  - Thread executes sequentially
  - Interruptible: processor can turn to another thread

- Thread switch
  - Switching processor between threads within same process
  - Typically less costly than process switch

In Java, a single task is called a thread. The term "thread" refers to a "thread of control" or "thread of execution," meaning a sequence of instructions that are executed one after another—the thread extends through time, connecting each instruction to the next.

In a multithreaded program, there can be many threads of control, weaving through time in parallel and forming the complete fabric of the program.

There are two ways to program a thread. One is to create a subclass of *Thread* and to define the method `public void run()` in the subclass.

```
public class NamedThread extends Thread {
   private String name;  // The name of this thread.
   public NamedThread(String name) {  // Constructor gives name to thread.
      this.name = name;
   }
   public void run() {  // The run method prints a message to standard output.
      System.out.println("Greetings from thread '" + name + "'!");
   }
}
```

To use a *NamedThread*, you must of course create an object belonging to this class. For example,

```
NamedThread greetings = new NamedThread("Fred");
```

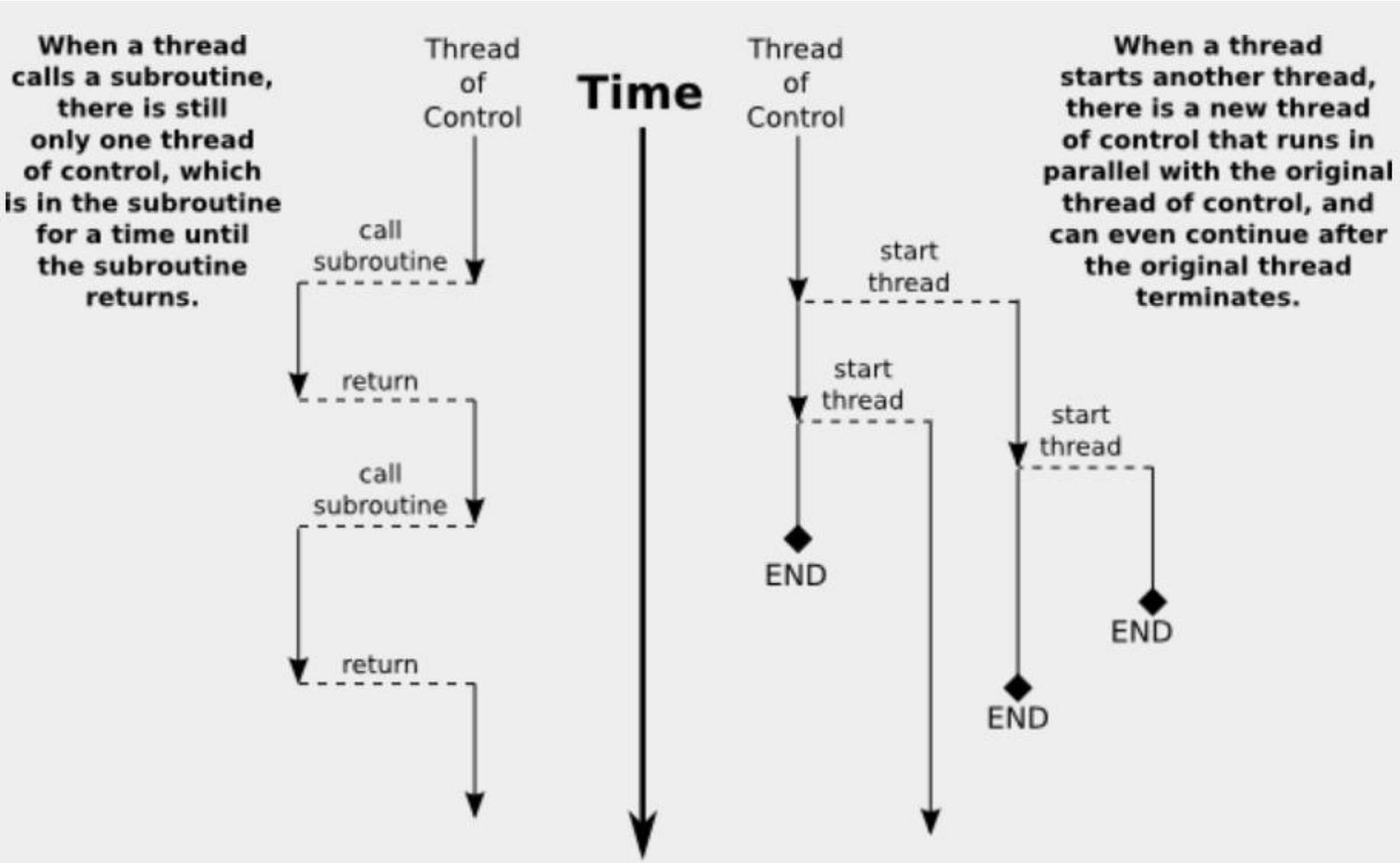
However, creating the object does not automatically start the thread running or cause its `run()` method to be executed.

```
greetings.start();
```

The start() method returns immediately after starting the new thread of control, **without waiting for the thread to terminate**.
This means that the code in the thread's run() method executes at the same time as the statements that follow the call to the start() method.

```
NamedThread greetings = new NamedThread("Fred");
greetings.start();
System.out.println("Thread has been started");
```

After greetings.start() is executed, there are two threads.
One of them will print "Thread has been started"
while the other one wants to print "Greetings from thread 'Fred'!".


When a thread calls a subroutine, there is still only one thread of control, which is in the subroutine for a time until the subroutine returns.
Thread of Control
Time
Thread of Control
When a thread starts another thread, there is a new thread of control that runs in parallel with the original thread of control, and can even continue after the original thread terminates.
call subroutine
start thread
return
start thread
start thread
call subroutine
END
END
return
END

The *Thread* class has a constructor that takes a *Runnable* as its parameter. When an object that implements the *Runnable* interface is passed to that constructor, the run() method of the thread will simply call the run() method from the *Runnable*, and calling the thread's start() method.

```
public class NamedRunnable implements Runnable {
   private String name;  // The name of this Runnable.
   public NamedRunnable(String name) {  // Constructor gives name to object.
      this.name = name;
   }
   public void run() {  // The run method prints a message to standard output.
      System.out.println("Greetings from runnable '" + name +"'!");
   }
}
```

To use this version of the class, we would create a *NamedRunnable* object and use that object to create an object of type *Thread*:

```
NamedRunnable greetings = new NamedRunnable("Fred");
Thread greetingsThread = new Thread(greetings);
greetingsThread.start();
```

Finally, a *Runnable* object can be given as a lambda expression.

In particular, the parameter to new Thread(r) can be a lambda expression.

```
Thread greetingsFromFred = new Thread(
    () -> System.out.println("Greetings from Fred!")
);
greetingsFromFred.start();
```

Running a thread is simple in Java—follow these steps:

1. Write a class that implements the `Runnable` interface. That interface has a single method called `run`:

```
public interface Runnable {
   void run();
}
```

2. Place the code for your task into the `run` method of your class:

```
public class MyRunnable implements Runnable {
   public void run() {
      Task statements.
      . . .
   }
}
```

3. Create an object of your runnable class:

```
Runnable r = new MyRunnable();
```

4. Construct a `Thread` object from the runnable object:

```
Thread t = new Thread(r);
```

5. Call the `start` method to start the thread:

```
t.start();
```

GreetingRunnable.java

```java
import java.util.Date;

/**
   A runnable that repeatedly prints a greeting.
*/
public class GreetingRunnable implements Runnable {
   private static final int REPETITIONS = 10;
   private static final int DELAY = 1000;

   private String greeting;

   /**
      Constructs the runnable object.
      @param aGreeting the greeting to display
   */
   public GreetingRunnable (String aGreeting) {
      greeting = aGreeting;
   }

   public void run () {
      try {
         for (int i = 1; i <= REPETITIONS; i++) {
            Date now = new Date();
            System.out.println( now + " " + greeting );
            Thread.sleep( DELAY );
         }
      } catch (InterruptedException exception) {}
   }
}
```

GreetingThreadRunner.java

```java
/**
   This program runs two greeting threads in parallel.
*/
public class GreetingThreadRunner {
   public static void main (String[] args) {
      GreetingRunnable r1 = new GreetingRunnable( "Hello" );
      GreetingRunnable r2 = new GreetingRunnable( "Goodbye" );
      Thread t1 = new Thread( r1 );
      Thread t2 = new Thread( r2 );
      t1.start();
      t2.start();
   }
}

```

```
H:\work\2021f\code-ch22-MultiThreading\section_1>javac GreetingThreadRunner.java

H:\work\2021f\code-ch22-MultiThreading\section_1>java GreetingThreadRunner
Thu Dec 02 14:01:22 CST 2021 Hello
Thu Dec 02 14:01:22 CST 2021 Goodbye
Thu Dec 02 14:01:23 CST 2021 Hello
Thu Dec 02 14:01:23 CST 2021 Goodbye
Thu Dec 02 14:01:24 CST 2021 Hello
Thu Dec 02 14:01:24 CST 2021 Goodbye
Thu Dec 02 14:01:25 CST 2021 Hello
Thu Dec 02 14:01:25 CST 2021 Goodbye
Thu Dec 02 14:01:26 CST 2021 Hello
Thu Dec 02 14:01:26 CST 2021 Goodbye
Thu Dec 02 14:01:27 CST 2021 Hello
Thu Dec 02 14:01:27 CST 2021 Goodbye
Thu Dec 02 14:01:28 CST 2021 Hello
Thu Dec 02 14:01:28 CST 2021 Goodbye
Thu Dec 02 14:01:29 CST 2021 Hello
Thu Dec 02 14:01:29 CST 2021 Goodbye
Thu Dec 02 14:01:30 CST 2021 Hello
Thu Dec 02 14:01:30 CST 2021 Goodbye
Thu Dec 02 14:01:31 CST 2021 Hello
Thu Dec 02 14:01:31 CST 2021 Goodbye

H:\work\2021f\code-ch22-MultiThreading\section_1>
```

Use the Runnable Interface

In Java, you can define the task statements of a thread in two ways. As you have seen already, you can place the statements into the run method of a class that implements the Runnable interface. Then you use an object of that class to construct a Thread object. You can also form a subclass of the Thread class, and place the task statements into the run method of your subclass:

```
public class MyThread extends Thread {
   public void run() {
      Task statements.
      . . .
   }
}
```

Then you construct an object of the subclass and call the start method:

```
Thread t = new MyThread();
t.start();
```

This approach is marginally easier than using a Runnable, and it also seems quite intuitive. However, if a program needs a large number of threads, or if a program executes in a resource-constrained device, such as a cell phone, it can be quite expensive to construct a separate thread for each task. Special Topic 22.1 shows how to use a *thread pool* to overcome this problem. A thread pool uses a small number of threads to execute a larger number of runnables.

The Runnable interface is designed to encapsulate the concept of a sequence of statements that can run in parallel with other tasks, without equating it with the concept of a thread, a potentially expensive resource that is managed by the operating system.

Thread Pools

A program that creates a huge number of short-lived threads can be inefficient. Threads are managed by the operating system, and there is a cost for creating threads. Each thread requires memory, and thread creation takes time. This cost can be reduced by using a *thread pool*. A thread pool creates a number of threads and keeps them alive. When you add a `Runnable` object to the thread pool, the next idle thread executes its `run` method.

For example, the following statements submit two runnables to a thread pool:

```
Runnable r1 = new GreetingRunnable("Hello");
Runnable r2 = new GreetingRunnable("Goodbye");
ExecutorService pool = Executors.newFixedThreadPool(MAX_THREADS);
pool.execute(r1);
pool.execute(r2);
```

If many runnables are submitted for execution, then the pool may not have enough threads available. In that case, some runnables are placed in a queue until a thread is idle. As a result, the cost of creating threads is minimized. However, the runnables that are run by a particular thread are executed sequentially, not in parallel.

Thread pools are particularly important for server programs, such as database and web servers, that repeatedly execute requests from multiple clients. Rather than spawning a new thread for each request, the requests are implemented as runnable objects and submitted to a thread pool.

Race Conditions

When threads share access to a common object, they can conflict with each other. To demonstrate the problems that can arise, we will investigate a sample program in which multiple threads manipulate a bank account.

We construct a bank account that starts out with a zero balance. We create two sets of threads:

- Each thread in the first set repeatedly deposits $100.
- Each thread in the second set repeatedly withdraws $100.

BankAccount.java

```java
/**
   A bank account has a balance that can be changed by
   deposits and withdrawals.
*/
public class BankAccount {
   private double balance;

   /**
      Constructs a bank account with a zero balance.
   */
   public BankAccount() {
      balance = 0;
   }

   /**
      Deposits money into the bank account.
      @param amount the amount to deposit
   */
   public void deposit (double amount) {
      System.out.print( "Depositing " + amount );
      double newBalance = balance + amount;
      System.out.println( ", new balance is " + newBalance );
      balance = newBalance;
   }

```

BankAccount.java

```
25
26      /**
27         Withdraws money from the bank account.
28         @param amount the amount to withdraw
29      */
30      public void withdraw (double amount) {
31         System.out.print("Withdrawing " + amount);
32         double newBalance = balance - amount;
33         System.out.println(", new balance is " + newBalance);
34         balance = newBalance;
35      }
36
37      /**
38         Gets the current balance of the bank account.
39         @return the current balance
40      */
41      public double getBalance() {
42         return balance;
43      }
44   }
45
```

DepositRunnable.java

```java
/**
   A deposit runnable makes periodic deposits to a bank account.
*/
public class DepositRunnable implements Runnable {
   private static final int DELAY = 1;
   private BankAccount account;
   private double amount;
   private int count;

   /**
      Constructs a deposit runnable.
      @param account the account into which to deposit money
      @param amount the amount to deposit in each repetition
      @param count the number of repetitions
   */
   public DepositRunnable (BankAccount account, double amount, int count) {
      this.account = account;
      this.amount = amount;
      this.count = count;
   }

   public void run() {
      try {
         for (int i = 1; i <= count; i++) {
            account.deposit( amount );
            Thread.sleep( DELAY );
         }
      } catch (InterruptedException exception) {}
   }
}
```

WithdrawRunnable.java

```java
/**
   A withdraw runnable makes periodic withdrawals from a bank account.
*/
public class WithdrawRunnable implements Runnable {
   private static final int DELAY = 1;
   private BankAccount account;
   private double amount;
   private int count;

   /**
      Constructs a withdraw runnable.
      @param account the account from which to withdraw money
      @param amount the amount to withdraw in each repetition
      @param count the number of repetitions
   */
   public WithdrawRunnable (BankAccount account, double amount, int count) {
      this.account = account;
      this.amount = amount;
      this.count = count;
   }

   public void run() {
      try {
         for (int i = 1; i <= count; i++) {
            account.withdraw( amount );
            Thread.sleep( DELAY );
         }
      } catch (InterruptedException exception) {}
   }
}
```

BankAccountThreadRunner.java

```java
/**
   This program runs threads that deposit and withdraw
   money from the same bank account.
*/
public class BankAccountThreadRunner {
   public static void main (String[] args) {
      BankAccount account = new BankAccount();
      final double AMOUNT = 100;
      final int REPETITIONS = 100;
      final int THREADS = 100;

      for (int i = 1; i <= THREADS; i++) {
         DepositRunnable d =
            new DepositRunnable( account, AMOUNT, REPETITIONS );
         WithdrawRunnable w =
            new WithdrawRunnable( account, AMOUNT, REPETITIONS );

         Thread dt = new Thread(d);
         Thread wt = new Thread(w);

         dt.start();
         wt.start();
      }
   }
}
```

result.txt

```
Withdrawing 100.0Depositing 100.0, new balance is 100.0
Depositing 100.0Withdrawing 100.0, new balance is 0.0
Depositing 100.0Depositing 100.0Withdrawing 100.0, new balance is -100.0
, new balance is -100.0
Withdrawing 100.0, new balance is -200.0
Withdrawing 100.0, new balance is -300.0
Withdrawing 100.0, new balance is -400.0
Depositing 100.0Depositing 100.0, new balance is -300.0
Depositing 100.0, new balance is -200.0
Withdrawing 100.0, new balance is -300.0
Withdrawing 100.0, new balance is -400.0
Withdrawing 100.0, new balance is -500.0
Withdrawing 100.0, new balance is -600.0
Withdrawing 100.0, new balance is -700.0
Withdrawing 100.0Withdrawing 100.0, new balance is -800.0
Withdrawing 100.0, new balance is -900.0
Withdrawing 100.0Depositing 100.0Withdrawing 100.0, new balance is -800.0
Withdrawing 100.0, new balance is -300.0
Depositing 100.0Depositing 100.0, new balance is -200.0
Withdrawing 100.0, new balance is 100.0
Withdrawing 100.0Depositing 100.0, new balance is 200.0
Withdrawing 100.0, new balance is 100.0
, new balance is 100.0
Depositing 100.0, new balance is 200.0
Depositing 100.0, new balance is 300.0
```

result.txt

```
19975   Depositing 100.0, new balance is -8700.0
19976   , new balance is -8900.0
19977   Withdrawing 100.0, new balance is -9000.0
19978   Withdrawing 100.0, new balance is -9100.0
19979   Withdrawing 100.0, new balance is -9200.0
19980   Withdrawing 100.0, new balance is -9300.0
19981   , new balance is -8700.0
19982   Depositing 100.0, new balance is -8600.0
19983   Depositing 100.0Depositing 100.0, new balance is -8500.0
19984   Withdrawing 100.0, new balance is -8600.0
19985   Withdrawing 100.0, new balance is -8700.0
19986   Withdrawing 100.0, new balance is -8800.0
19987   Depositing 100.0, new balance is -8700.0
19988   , new balance is -8500.0
19989   Withdrawing 100.0, new balance is -8600.0
19990   Withdrawing 100.0, new balance is -8700.0
19991   Depositing 100.0, new balance is -8600.0
19992   Depositing 100.0, new balance is -8500.0
19993   Withdrawing 100.0, new balance is -8600.0
19994   Withdrawing 100.0, new balance is -8700.0
19995   Depositing 100.0, new balance is -8600.0
19996   Depositing 100.0, new balance is -8500.0
19997   Depositing 100.0, new balance is -8400.0
19998   Depositing 100.0, new balance is -8300.0
19999   Depositing 100.0, new balance is -8200.0
20000   Depositing 100.0, new balance is -8100.0
20001
```

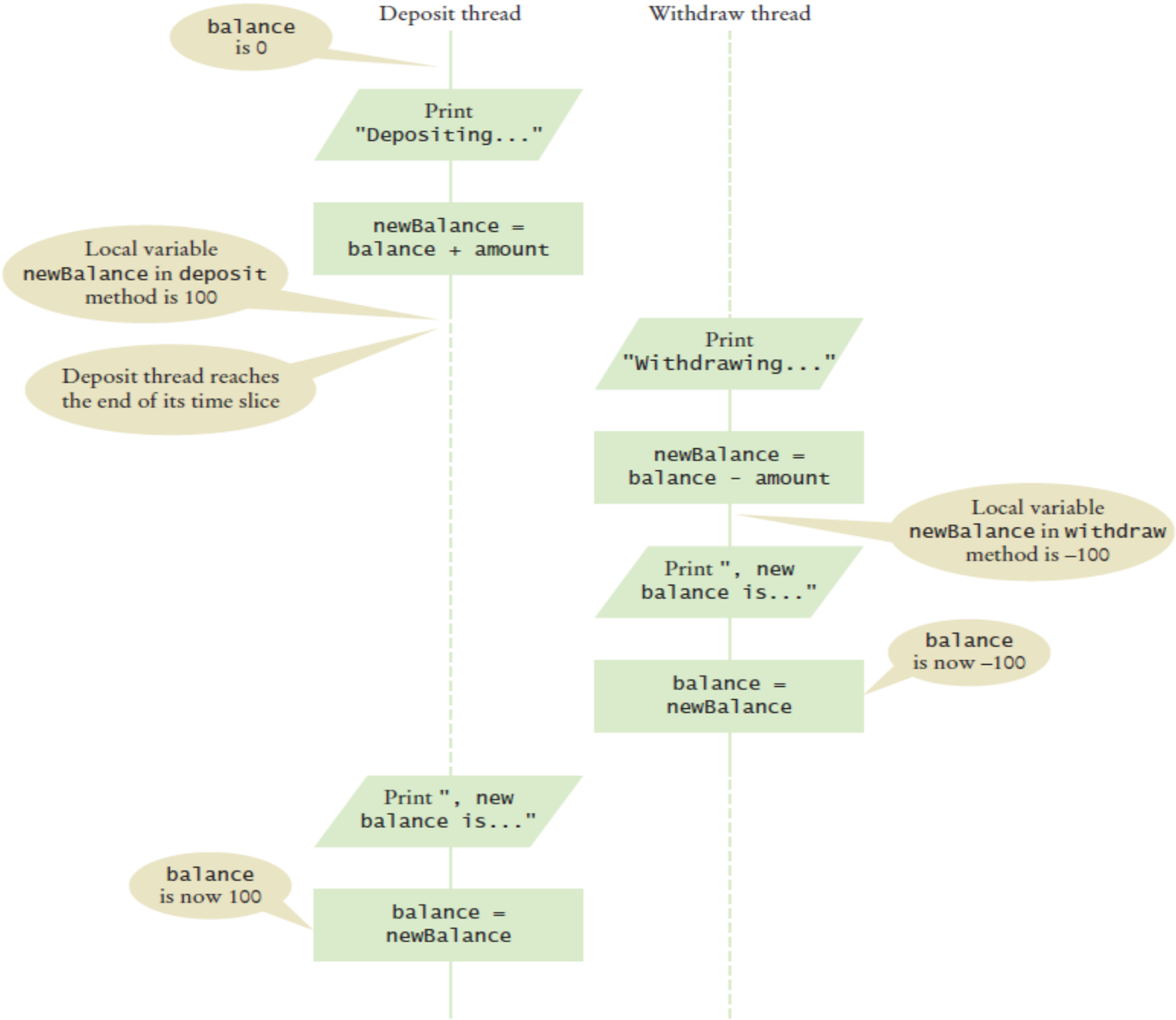


Figure 1 Corrupting the Contents of the `balance` Variable

Synchronizing Object Access

To solve problems such as the one that you observed in the preceding section, use a **lock object**. The lock object is used to control the threads that want to manipulate a shared resource.

The Java library defines a `Lock` interface and several classes that implement this interface. The `ReentrantLock` class is the most commonly used lock class, and the only one that we cover in this book. (**Locks** are a feature added in Java version 5.0. Earlier versions of Java have a lower-level facility for thread synchronization—see Special Topic 22.2).

Typically, a lock object is added to a class whose methods access shared resources, like this:

```
public class BankAccount {

   private Lock balanceChangeLock;
   . . .
   public BankAccount() {

      balanceChangeLock = new ReentrantLock();
      . . .
   }
}
```

All code that manipulates the shared resource is surrounded by calls to lock and unlock the lock object:

```
balanceChangeLock.lock();
Manipulate the shared resource.
balanceChangeLock.unlock();
```

However, this sequence of statements has a potential flaw. If the code between the calls to `lock` and `unlock` throws an exception, the call to `unlock` never happens. This is a serious problem. After an exception, the current thread continues to hold the lock, and no other thread can acquire it. To overcome this problem, place the call to `unlock` into a `finally` clause.

For example, here is the code for the `deposit` method:

```
public void deposit (double amount) {
   balanceChangeLock.lock();
   try {
      System.out.print("Depositing " + amount);
      double newBalance = balance + amount;
      System.out.println(", new balance is " + newBalance);
      balance = newBalance;
   } finally {
      balanceChangeLock.unlock();
   }
}
```

Figure 2
Visualizing Object Locks

Creatas/Punchstock.

Avoiding Deadlocks

You can use lock objects to ensure that shared data are in a consistent state when several threads access them. However, locks can lead to another problem. It can happen that one thread acquires a lock and then waits for another thread to do some essential work. If that other thread is currently waiting to acquire the same lock, then neither of the two threads can proceed. Such a situation is called a **deadlock** or **deadly embrace**.

A deadlock occurs if no thread can proceed because each thread is waiting for another to do some work first.

Suppose we want to disallow negative bank balances in our program. Here's a naive way of doing that. In the `run` method of the `WithdrawRunnable` class, we can check the balance before withdrawing money:

```
if (account.getBalance() >= amount) {
   account.withdraw(amount);
}
```

Clearly, the test should be moved inside the `withdraw` method. That ensures that the test for sufficient funds and the actual withdrawal cannot be separated. Thus, the `withdraw` method could look like this:

```
public void withdraw (double amount) {
   balanceChangeLock.lock();
   try {
      while (balance < amount) {
         Wait for the balance to grow.
      }
      . . .
   } finally {
      balanceChangeLock.unlock();
   }
}
```

To overcome this problem, we use a **condition object**. Condition objects allow a thread to temporarily release a lock, so that another thread can proceed, and to regain the lock at a later time.

Each condition object belongs to a specific lock object. You obtain a condition object with the `newCondition` method of the `Lock` interface. For example,

```
public class BankAccount {
   private Lock balanceChangeLock;
   private Condition sufficientFundsCondition;
   . . .
   public BankAccount() {
      balanceChangeLock = new ReentrantLock();
      sufficientFundsCondition = balanceChangeLock.newCondition();
      . . .
   }
}
```

It is customary to give the condition object a name that describes the condition that you want to test (such as "sufficient funds"). You need to implement an appropriate test. For as long as the test is not fulfilled, call the `await` method on the condition object:

```
public void withdraw (double amount) {
   balanceChangeLock.lock();
   try {
      while (balance < amount) {
         sufficientFundsCondition.await();
      }
      . . .
   } finally {
      balanceChangeLock.unlock();
   }
}
```

When a thread calls `await`, it is not simply deactivated in the same way as a thread that reaches the end of its time slice. Instead, it is in a blocked state, and it will not be activated by the thread scheduler until it is unblocked. To unblock, another thread must execute the `signalAll` method *on the same condition object*. The `signalAll` method unblocks all threads waiting on the condition. They can then compete with all other threads that are waiting for the lock object. Eventually, one of them will gain access to the lock, and it will exit from the `await` method.

In our situation, the `deposit` method calls `signalAll`:

```
public void deposit (double amount) {
   balanceChangeLock.lock();
   try {

      . . .
      sufficientFundsCondition.signalAll();
   } finally {
      balanceChangeLock.unlock();
   }
}
```

The call to `signalAll` notifies the waiting threads that sufficient funds *may be* available, and that it is worth testing the loop condition again.

BankAccount.java

```
1   import java.util.concurrent.locks.Condition;
2   import java.util.concurrent.locks.Lock;
3   import java.util.concurrent.locks.ReentrantLock;
4
5   /**
6      A bank account has a balance that can be changed by
7      deposits and withdrawals.
8   */
9   public class BankAccount {
10     private double balance;
11     private Lock balanceChangeLock;
12     private Condition sufficientFundsCondition;
13
14     /**
15        Constructs a bank account with a zero balance.
16     */
17     public BankAccount() {
18        balance = 0;
19        balanceChangeLock = new ReentrantLock();
20        sufficientFundsCondition = balanceChangeLock.newCondition();
21     }
22
```

BankAccount.java

```
22
23      /**
24         Deposits money into the bank account.
25         @param amount the amount to deposit
26      */
27      public void deposit (double amount) {
28         balanceChangeLock.lock();
29         try {
30            System.out.print( "Depositing " + amount );
31            double newBalance = balance + amount;
32            System.out.println( ", new balance is " + newBalance );
33            balance = newBalance;
34            sufficientFundsCondition.signalAll();
35         } finally {
36            balanceChangeLock.unlock();
37         }
38      }
39
```

BankAccount.java

```java
    /**
       Withdraws money from the bank account.
       @param amount the amount to withdraw
    */
    public void withdraw (double amount) throws InterruptedException {
       balanceChangeLock.lock();
       try {
          while (balance < amount) {
             sufficientFundsCondition.await();
          }
          System.out.print( "Withdrawing " + amount );
          double newBalance = balance - amount;
          System.out.println( ", new balance is " + newBalance );
          balance = newBalance;
       } finally {
          balanceChangeLock.unlock();
       }
    }

    /**
       Gets the current balance of the bank account.
       @return the current balance
    */
    public double getBalance() {
       return balance;
    }
}
```

result.txt

```
Depositing 100.0, new balance is 100.0
Withdrawing 100.0, new balance is 0.0
Depositing 100.0, new balance is 100.0
Withdrawing 100.0, new balance is 0.0
Depositing 100.0, new balance is 100.0
Withdrawing 100.0, new balance is 0.0
Depositing 100.0, new balance is 100.0
Withdrawing 100.0, new balance is 0.0
Depositing 100.0, new balance is 100.0
Withdrawing 100.0, new balance is 0.0
Depositing 100.0, new balance is 100.0
Depositing 100.0, new balance is 200.0
Withdrawing 100.0, new balance is 100.0
Depositing 100.0, new balance is 200.0
Withdrawing 100.0, new balance is 100.0
Depositing 100.0, new balance is 200.0
Depositing 100.0, new balance is 300.0
Withdrawing 100.0, new balance is 200.0
Depositing 100.0, new balance is 300.0
Depositing 100.0, new balance is 400.0
Withdrawing 100.0, new balance is 300.0
Withdrawing 100.0, new balance is 200.0
Depositing 100.0, new balance is 300.0
Withdrawing 100.0, new balance is 200.0
Withdrawing 100.0, new balance is 100.0
Depositing 100.0, new balance is 200.0
```

result.txt

```
19975   Withdrawing 100.0, new balance is 900.0
19976   Withdrawing 100.0, new balance is 800.0
19977   Withdrawing 100.0, new balance is 700.0
19978   Depositing 100.0, new balance is 800.0
19979   Withdrawing 100.0, new balance is 700.0
19980   Withdrawing 100.0, new balance is 600.0
19981   Depositing 100.0, new balance is 700.0
19982   Depositing 100.0, new balance is 800.0
19983   Depositing 100.0, new balance is 900.0
19984   Withdrawing 100.0, new balance is 800.0
19985   Depositing 100.0, new balance is 900.0
19986   Depositing 100.0, new balance is 1000.0
19987   Withdrawing 100.0, new balance is 900.0
19988   Withdrawing 100.0, new balance is 800.0
19989   Withdrawing 100.0, new balance is 700.0
19990   Withdrawing 100.0, new balance is 600.0
19991   Depositing 100.0, new balance is 700.0
19992   Withdrawing 100.0, new balance is 600.0
19993   Depositing 100.0, new balance is 700.0
19994   Withdrawing 100.0, new balance is 600.0
19995   Withdrawing 100.0, new balance is 500.0
19996   Withdrawing 100.0, new balance is 400.0
19997   Withdrawing 100.0, new balance is 300.0
19998   Withdrawing 100.0, new balance is 200.0
19999   Withdrawing 100.0, new balance is 100.0
20000   Withdrawing 100.0, new balance is 0.0
20001
```

Object Locks and Synchronized Methods

The `Lock` and `Condition` interfaces were added in Java version 5.0. They overcome limitations of the thread synchronization mechanism in earlier Java versions. In this note, we discuss that classic mechanism.

Every Java object has one built-in lock and one built-in condition variable. The lock works in the same way as a `ReentrantLock` object. However, to acquire the lock, you call a **synchronized method**.

You simply tag all methods that contain thread-sensitive code (such as the `deposit` and `withdraw` methods of the `BankAccount` class) with the `synchronized` reserved word.

```
public class BankAccount
{
   public synchronized void deposit(double amount)
   {
      System.out.print("Depositing " + amount);
      double newBalance = balance + amount;
      System.out.println(", new balance is " + newBalance);
      balance = newBalance;
   }

   public synchronized void withdraw(double amount)
   {
      . . .
   }
   . . .
}
```

In other words, the `synchronized` reserved word automatically implements the `lock/try/finally/unlock` idiom for the built-in lock.

The object lock has a single condition variable that you manipulate with the `wait`, `notifyAll`, and `notify` methods of the `Object` class. If you call `x.wait()`, the current thread is added to the set of threads that is waiting for the condition of the object `x`. Most commonly, you will call `wait()`, which makes the current thread wait on `this`. For example,

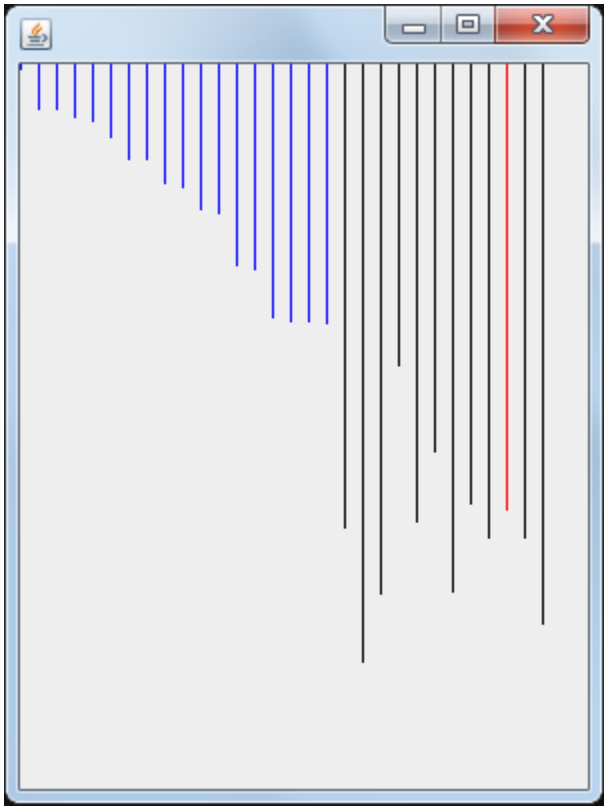
```
public synchronized void withdraw(double amount)
      throws InterruptedException
{
   while (balance < amount)
   {
      wait();
   }
   . . .
}
```

The call `notifyAll()` unblocks all threads that are waiting for `this`:

```
public synchronized void deposit(double amount)
{
   . . .
   notifyAll();
}
```

Application: Algorithm Animation

One popular use for thread programming is animation. A program that displays an animation shows different objects moving or changing in some way as time progresses. This is often achieved by launching one or more threads that compute how parts of the animation change.



Thus, the algorithm state is described by three items:

- The array of values
- The size of the already sorted area
- The currently marked element

We add this state to the `SelectionSorter` class.

```
public class SelectionSorter
{
   // This array is being sorted
   private int[] a;
   // These instance variables are needed for drawing
   private int markedPosition = -1;
   private int alreadySorted = -1;
   . . .
}
```

The array that is being sorted is now an instance variable, and we will change the `sort` method from a static method to an instance method.

This state is accessed by two threads: the thread that sorts the array and the thread that paints the frame. We use a lock to synchronize access to the shared state.

Finally, we add a `component` instance variable to the algorithm class and augment the constructor to set it. That instance variable is needed for repainting the component and finding out the dimensions of the component when drawing the algorithm state.

```
public class SelectionSorter
{
   private JComponent component;
   . . .
   public SelectionSorter(int[] anArray, JComponent aComponent)
   {
      a = anArray;
      sortStateLock = new ReentrantLock();
      component = aComponent;
   }
}
```

At each point of interest, the algorithm needs to pause so that the user can admire the graphical output. We supply the `pause` method shown below, and call it at various places in the algorithm. The `pause` method repaints the component and sleeps for a small delay that is proportional to the number of steps involved.

```
public void pause(int steps) throws InterruptedException
{
   component.repaint();
   Thread.sleep(steps * DELAY);
}
```

We add a `draw` method to the algorithm class that can draw the current state of the data structure, with the items of special interest highlighted. The `draw` method is specific to the particular algorithm. This `draw` method draws the array elements as a sequence of sticks in different colors. The already sorted portion is blue, the marked position is red, and the remainder is black .

```
/**
   Draws the current state of the sorting algorithm.
   @param g the graphics context
*/
public void draw (Graphics g) {
   sortStateLock.lock();
   try {
      int deltaX = component.getWidth() / a.length;
      for (int i = 0; i < a.length; i++) {
         if (i == markedPosition) {
            g.setColor( Color.RED);
         } else if (i <= alreadySorted) {
            g.setColor( Color.BLUE);
         } else {
            g.setColor( Color.BLACK);
         }
         g.drawLine( i * deltaX, 0, i * deltaX, a[i]);
      }
   } finally {
      sortStateLock.unlock();
   }
}
```

SelectionSortViewer.java

```
1   import java.awt.BorderLayout;
2   import javax.swing.JButton;
3   import javax.swing.JFrame;
4
5   public class SelectionSortViewer {
6      public static void main (String[] args) {
7         JFrame frame = new JFrame();
8
9         final int FRAME_WIDTH = 300;
10        final int FRAME_HEIGHT = 400;
11
12        frame.setSize( FRAME_WIDTH, FRAME_HEIGHT);
13        frame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE);
14
15        final SelectionSortComponent component =
16           new SelectionSortComponent();
17        frame.add( component, BorderLayout.CENTER);
18
19        frame.setVisible( true);
20        component.startAnimation();
21     }
22  }
23
```

SelectionSortComponent.java

```java
import java.awt.Graphics;
import javax.swing.JComponent;

/**
   A component that displays the current state of the selection sort algorithm.
*/
public class SelectionSortComponent extends JComponent {
   private SelectionSorter sorter;

   /**
      Constructs the component.
   */
   public SelectionSortComponent() {
      int[] values = ArrayUtil.randomIntArray( 30, 300 );
      sorter = new SelectionSorter( values, this );
   }

   public void paintComponent (Graphics g) {
      sorter.draw(g);
   }

```

SelectionSortComponent.java

```
21
22      /**
23         Starts a new animation thread.
24      */
25      public void startAnimation() {
26         class AnimationRunnable implements Runnable {
27            public void run() {
28               try {
29                  sorter.sort();
30               } catch (InterruptedException exception) {}
31            }
32         }
33
34         Runnable r = new AnimationRunnable();
35         Thread t = new Thread(r);
36         t.start();
37      }
38   }
39
```

ArrayUtil.java

```java
import java.util.Random;

/**
   This class contains utility methods for array manipulation.
*/
public class ArrayUtil {
   private static Random generator = new Random();

   /**
      Creates an array filled with random values.
      @param length the length of the array
      @param n the number of possible random values
      @return an array filled with length numbers between
      0 and n - 1
   */
   public static int[] randomIntArray (int length, int n) {
      int[] a = new int[length];
      for (int i = 0; i < a.length; i++) {
         a[i] = generator.nextInt(n);
      }

      return a;
   }

```

ArrayUtil.java

```
24
25      /**
26         Swaps two entries of an array.
27         @param a the array
28         @param i the first position to swap
29         @param j the second position to swap
30      */
31      public static void swap (int[] a, int i, int j) {
32         int temp = a[i];
33         a[i] = a[j];
34         a[j] = temp;
35      }
36   }
37
```

SelectionSorter.java

```
1  import java.awt.Color;
2  import java.awt.Graphics;
3  import java.util.concurrent.locks.Lock;
4  import java.util.concurrent.locks.ReentrantLock;
5  import javax.swing.JComponent;
6
7  /**
8     This class sorts an array, using the selection sort algorithm.
9  */
10 public class SelectionSorter {
11    private int[] a;    // This array is being sorted
12    // These instance variables are needed for drawing
13    private int markedPosition = -1;
14    private int alreadySorted = -1;
15
16    private Lock sortStateLock;
17
18    // The component is repainted when the animation is paused
19    private JComponent component;
20
21    private static final int DELAY = 100;
22
```

SelectionSorter.java

```
23    /**
24       Constructs a selection sorter.
25       @param anArray the array to sort
26       @param aComponent the component to be repainted when the animation
27       pauses
28    */
29    public SelectionSorter (int[] anArray, JComponent aComponent) {
30       a = anArray;
31       sortStateLock = new ReentrantLock();
32       component = aComponent;
33    }
34
35    /**
36       Sorts the array managed by this selection sorter.
37    */
38    public void sort() throws InterruptedException {
39       for (int i = 0; i < a.length - 1; i++) {
40          int minPos = minimumPosition(i);
41          sortStateLock.lock();
42          try {
43             ArrayUtil.swap( a, minPos, i );
44             alreadySorted = i;   // For animation
45          } finally {
46             sortStateLock.unlock();
47          }
48          pause( 2 );
49       }
50    }
51
```

SelectionSorter.java

```
51
52      /**
53         Finds the smallest element in a tail range of the array
54         @param from the first position in a to compare
55         @return the position of the smallest element in the
56         range a[from]...a[a.length - 1]
57      */
58      private int minimumPosition (int from) throws InterruptedException {
59         int minPos = from;
60         for (int i = from + 1; i < a.length; i++) {
61            sortStateLock.lock();
62            try {
63               if (a[i] < a[minPos]) { minPos = i; }
64               markedPosition = i;   // For animation
65            } finally {
66               sortStateLock.unlock();
67            }
68            pause( 2 );
69         }
70         return minPos;
71      }
72
```

SelectionSorter.java

```
73    /**
74       Draws the current state of the sorting algorithm.
75       @param g the graphics context
76    */
77    public void draw (Graphics g) {
78       sortStateLock.lock();
79       try {
80          int deltaX = component.getWidth() / a.length;
81          for (int i = 0; i < a.length; i++) {
82             if (i == markedPosition) {
83                g.setColor( Color.RED);
84             } else if (i <= alreadySorted) {
85                g.setColor( Color.BLUE);
86             } else {
87                g.setColor( Color.BLACK);
88             }
89             g.drawLine( i * deltaX, 0, i * deltaX, a[i]);
90          }
91       } finally {
92          sortStateLock.unlock();
93       }
94    }
95
```

SelectionSorter.java

```
95
96      /**
97         Pauses the animation.
98         @param steps the number of steps to pause
99      */
100     public void pause (int steps) throws InterruptedException {
101        component.repaint();
102        Thread.sleep( steps * DELAY);
103     }
104  }
105
```